Programming Paradigms

BSc Computer Science - Durham University

Matthew Emerson

November 2026

Contents

1 Module Introduction **1**
1.1 Assessment . . . 1
1.2 Timetable . . . 1

2 Systems Programming **1**
2.1 Version Control and UNIX . . . 1
2.2 The Basics of Shell . . . 2
2.3 The Basics of C . . . 2

3 Functional Programming **4**

4 Object Oriented Programming **4**
4.1 Java Fundamentals . . . 4
4.2 OOP with Classes and Objects . . . 4
4.3 Encapsulation . . . 4
4.4 Inheritance . . . 4
4.5 Abstraction with Abstract Classes . . . 4
4.6 Abstraction with Interfaces . . . 4
4.7 Polymorphism . . . 4
4.8 Java Collections . . . 4
4.9 Best Practises . . . 4

I Appendix **5**

5 Bibliography **5**
5.1 Module Professors . . . 5
5.2 References . . . 5
5.3 List of Figures . . . 5
5.4 Reading List . . . 5

1 Module Introduction

Programming Paradigms has the module code COMP2221 and is led by Dr Stuart James.

1.1 Assessment

This module is assessed through one piece of summative coursework and two final exams, these can be seen in Table 1. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Systems Programming	*12th Oct*	*12th Nov*	*10th Dec*	*Dr Stuart James*	-
Systems Programming	23rd Oct	10th Dec	21st Jan	Dr Stuart James	50%
Object Oriented & Functional	May Exam	-	-	Ivkovic & Moses	50%

Table 1: Assessments for Programming Paradigms.

1.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 2.

Day	Time	Class Type	Location	Lecturer
Monday	11:00 - 12:00	Lecture	CLC202	Dr Stuart James
Monday	16:00 - 18:00	Computer Class	CB-LG001	n/a
Friday	14:00 - 15:00	Lecture	CLC202	Dr Stuart James

Table 2: Michaelmas Term Timetable for Programming Paradigms.

2 Systems Programming

This section is taught by Dr Stuart James. The reading list for this module is as follows:

- Bryant, R. and O'Hallaron, D. (2015). Computer systems: A Programmer's Perspective. 3rd edn. Pearson Education, Limited.

I have not made notes on the majority of this module as most of it is simply learning C. Where lecture contents differ from this, I have made notes.

Definition 2.0.0.1: Systems programming involves the development of individual pieces of software that allow the entire system to function as a single unit.

2.1 Version Control and UNIX

2.1.1 Introduction to UNIX

Files are stored in a hierarchical structure, allowing for grouping. To navigate around these files, we can use UNIX navigation commands as seen in Table 3.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Action	Command
List the contents of the current folder.	*ls*
Change folder/directory.	*cd*
Make new folder/directory.	*mkdir*
Move a file/folder.	*mv*
Copy a file/folder.	*cp*
Delete a file or, by adding $-r$, a directory.	*rm*
Check file/folder size.	*du*
List all files.	*find*

Table 3: UNIX file navigation commands.

2.2 The Basics of Shell

All shell files are saved with a *.sh* extension and must begin with #!*/bin/bash*.

2.2.1 Basic Commands

```
#!/bin/bash
for f in *
do
    #something in here
    echo $f
done
```

Algorithm 1: For loop example.

```
./myscript.sh 10 2

#!/bin/bash
if [$1 -lt $2]
then
    echo "yes" $1 "is less than" $2
else
    echo "no it isn't"
fi
```

Algorithm 2: If statement example (compares 10 and 2).

2.3 The Basics of C

2.3.1 Basic File Setup

Definition 2.3.1.1: Pre-processor directives are lines which start with a # and are commands to the C pre-processor.

- If the directive is #include, and <> are used, then */usr/include* is prioritised in UNIX by convention, whereas if "" are used, then the current directory is prioritised.
- **Definition 2.3.1.1**: If the directive is #define, then this is a definition macro. These are used to provide definitions in code.

- **Definition 2.3.1.2**: You can also use directives for debugging without generating overheads, these are called conditionals. They are as shown in Table 4.

Conditional	Function
#ifdef VAR	Tests if VAR is #defined.
#ifndef VAR	Tests if VAR is not #defined.
#else	Runs if the original condition evaluates as false.
#endif	Ends the if testing condition.

Table 4: Different types of conditionals.

You can also put debugging code in the preamble, as seen in Algorithm 3.

```
#define MY_DEBUG // Define an identifier.

#ifdef MY_DEBUG
    assert(i > 0);
    printf("i is %d \n", i);
#endif
```

Algorithm 3: Debugging Code Example.

Definition 2.3.1.4: You can also use #define to define parameterised macros. These act like functions, and an example can be seen in Algorithm 4.

```
#define ADD(a,b) a + b
```

Algorithm 4: Parameterised Macro Example.

Definition 2.3.1.5: The main() function is the entry function for C programs. You can only have one of these.

2.3.2 Printf Parameters

Value	Resulting Format
$\%d$	Signed decimal (int).
$\%u$	Unsigned decimal.
$\%o, \%x$	Octal and hexidecimal.
$\%l$	Long.
$\%f$	Floating point, so $\%1.2f$ will give 3.14.
$\%e$	Floating point (exponent form).
$\%c, \%s$	Character and string.

Table 5: First parameter of printf.

2.3.3 Variables

Each variable in C has one of the following four storage types.

- **Definition 2.3.3.1**: extern: When a variable is defined, it is allocated storage. When a variable is declared, the compiler knows a variable of a given type exists. Top-level variables default to extern storage class. They last for the lifetime and scope of the whole program.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- **Definition 2.3.3.2**: static: Static variables have the same lifetime as the program, static global variables have file scope, and static local variables have function scope. A static variable inside a function keeps its value between invocations.
- **Definition 2.3.3.3**: auto: Auto variables have the same lifetime as the function where they are defined. They have function scope and function lifetime.
- **Definition 2.3.3.4**: register: Suggests that a variable should be stored in a register rather than in main memory. It cannot use the address of (&) operator.

3 Functional Programming

This section is taught by Dr Billy Moses Jr.. Check for recommended reading.

4 Object Oriented Programming

This section is taught by Dr Mladen Ivkovic. Check for recommended reading.

4.1 Java Fundamentals

4.2 OOP with Classes and Objects

4.3 Encapsulation

4.4 Inheritance

4.5 Abstraction with Abstract Classes

4.6 Abstraction with Interfaces

4.7 Polymorphism

4.8 Java Collections

4.9 Best Practises

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part I
Appendix

5 Bibliography

5.1 Module Professors

Module Lead: Doctor Stuart James

- Focus: Artificial intelligence, computer vision, human computer interaction, and digital humanities.
- Email: stuart.a.james@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Billy Moses Jr.

- Focus: Algorithms and biologically inspired computing.
- Email: william.k.moses-jr@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Mladen Ivkcovic

- Focus: Performance prediction and optimisation.
- Email: mladen.ivkovic@durham.ac.uk
- Office Hours: Email to make an appointment.

5.2 References

5.3 List of Figures

5.4 Reading List

5.4.1 Systems Programming

Bryant, R. and O'Hallaron, D. (2015). Computer systems: A Programmer's Perspective. 3rd edn. Pearson Education, Limited.